Nama : Elyza Putri Rahayu

No Absen : 08

NIM : 254107020035

Kelas : TI – 1G

LAPORAN JOBSHEET PRAKTIKUM ALGORITMA DAN STRUKTUR DATA

PERTEMUAN 11

PERCOBAAN 1

```
PS D:\Algoritma-dan-Struktur-Data>  d:; cd 'd:\Algoritma-dan-St
ruktur-Data'; & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-X
X:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\elyza\Ap
pData\Roaming\Code\User\workspaceStorage\8b371f4b20fd407f29373e
42b230f640\redhat.java\jdt_ws\Algoritma-dan-Struktur-Data_9d581
4e9\bin' 'Pertemuan12.SLLMain08'
Linked List Kosong
Isi Linked List:
Dirga    21212203       4D      3.6

Isi Linked List:
Dirga    21212203       4D      3.6
Alvaro   24212200       1A      4.0

Isi Linked List:
Dirga    21212203       4D      3.6
Cintia   22212202       3C      3.5
Bimon    23212201       2B      3.8
Alvaro   24212200       1A      4.0
```

Kode Program Mahasiswa08 :

package *P*ERTEMUAN11;

import org.w3c.dom.Node;

public class Mahasiswa08 {

    String nama;

    String nim;

```
    String kelas;
    double ipk;

    Mahasiswa08() {

    }

    Mahasiswa08(String nm, String name, String kls, double ip) {
        nama = name;
        nim = nm;
        kelas = kls;
        ipk = ip;
    }

    void tampilInformasi() {
        System.out.println(nama + "\t " + nim + "\t" + kelas + "\t" + ipk);
    }
}
```

Kode Program NodeMahasiswa08 :

```
package PERTEMUAN11;

public class NodeMahasiswa08 {
    Mahasiswa08 data;
```

```
    NodeMahasiswa08 next;

    public NodeMahasiswa08(Mahasiswa08 data, NodeMahasiswa08 next) {
        this.data = data;
        this.next = next;
    }
}
```

Kode Program SingleLinkedList08 :

```
package PERTEMUAN11;

public class SingleLinkedList08 {
    NodeMahasiswa08 head;
    NodeMahasiswa08 tail;

    boolean IsEmpty() {
        return (head == null);
    }

    public void print() {
        if (!IsEmpty()) {
            NodeMahasiswa08 tmp = head;
            System.out.println("Isi Linked List:\t");
            while (tmp != null) {
```

```
            tmp.data.tampilInformasi();
            tmp = tmp.next;
        }
        System.out.println("");
    } else {
        System.out.println("Linked List Kosong");
    }
}

public void addFirst(Mahasiswa08 input) {
    NodeMahasiswa08 ndInput = new NodeMahasiswa08(input, null);
    if (IsEmpty()) {
        head = ndInput;
        tail = ndInput;
    } else {
        ndInput.next = head;
        head = ndInput;
    }
}

public void addLast(Mahasiswa08 input) {
    NodeMahasiswa08 ndInput = new NodeMahasiswa08(input, null);
    if (IsEmpty()) {
        head = ndInput;
```

```
        tail = ndInput;
    } else {
        tail.next = ndInput;
        tail = ndInput;
    }
}

public void insertAfter(String key, Mahasiswa08 input) {
    NodeMahasiswa08 ndInput = new NodeMahasiswa08(input, null);
    NodeMahasiswa08 temp = head;
    do {
        if (temp.data.nama.equalsIgnoreCase(key)) {
            ndInput.next = temp.next;
            temp.next = ndInput;
            if (ndInput.next == null) {
                tail = ndInput;
            }
            break;
        }
        temp = temp.next;
    } while (temp != null);
}

public void insertAt(int index, Mahasiswa08 input) {
```

```
        if (index < 0) {
            System.out.println("indeks salah");
        } else if (index == 0) {
            addFirst(input);
        } else {
            NodeMahasiswa08 temp = head;
            for (int i = 0; i < index - 1; i++) {
                temp = temp.next;
            }
            temp.next = new NodeMahasiswa08(input, temp.next);
            if (temp.next.next == null) {
                tail = temp.next;
            }
        }
    }
}
```

Kode Program SLLMain08 :

```
Package PERTEMUAN11;


public class SLLMain08 {
    public static void main(String[] args) {
        SingleLinkedList08 sll = new SingleLinkedList08();
```

```
    }

        Mahasiswa08 mhs1 = new Mahasiswa08("24212200", "Alvaro", "1A", 4.0);
        Mahasiswa08 mhs2 = new Mahasiswa08("23212201", "Bimon", "2B", 3.8);
        Mahasiswa08 mhs3 = new Mahasiswa08("22212202", "Cintia", "3C", 3.5);
        Mahasiswa08 mhs4 = new Mahasiswa08("21212203", "Dirga", "4D", 3.6);


        sll.print();
        sll.addFirst(mhs4);
        sll.print();
        sll.addLast(mhs1);
        sll.print();
        sll.insertAfter("Dirga", mhs3);
        sll.insertAt(2, mhs2);
        sll.print();
    }
}
```

Pertanyaan

1. Mengapa hasil compile kode program di baris pertama menghasilkan “Linked List Kosong”?

   Jawab : Karena pada saat program pertama kali dijalankan, linked list masih belum berisi data apa pun. Di dalam class SingleLinkedList08, kondisi kosong dicek di method:

   ```
   boolean IsEmpty() {
     return (head == null);
   }
   ```

   Nah, waktu object sll baru dibuat:

   ```
   SingleLinkedList08 sll = new SingleLinkedList08();
   ```

nilai head = null, artinya belum ada node sama sekali.

Terus kamu langsung panggil:

sll.print();

Maka program masuk ke bagian ini:

```
if (!IsEmpty()) {
  ...
} else {
  System.out.println("Linked List Kosong");
}
```

Karena masih kosong → otomatis keluar tulisan "Linked List Kosong".

2. Jelaskan kegunaan variable temp secara umum pada setiap method!

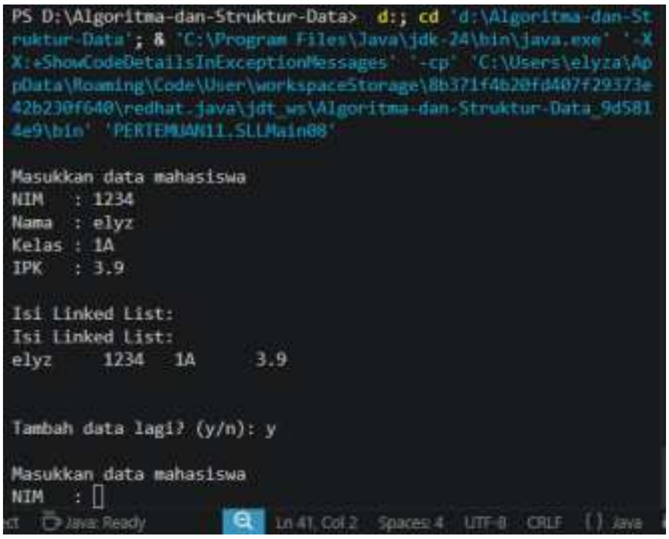
   Jawab : Variabel temp itu fungsinya sebagai penunjuk sementara (penjelajah) di dalam linked list.

   fungsi temp:

- Untuk menelusuri isi linked list
- Untuk mencari node tertentu
- Untuk menyisipkan atau memodifikasi node tanpa merusak head

   Kalau tidak pakai temp, kita bisa kehilangan referensi ke data awal (head), jadi program bisa error.

3. Lakukan modifikasi agar data dapat ditambahkan dari keyboard!



public class SLLMain08 {

```
}

public static void main(String[] args) {
    SingleLinkedList08 sll = new SingleLinkedList08();
    Scanner sc = new Scanner(System.in);

    String lanjut = "y";

    while (lanjut.equalsIgnoreCase("y")) {

        System.out.println("\nMasukkan data mahasiswa");

        System.out.print("NIM   : ");
        String nim = sc.nextLine();

        System.out.print("Nama  : ");
        String nama = sc.nextLine();

        System.out.print("Kelas : ");
        String kelas = sc.nextLine();

        System.out.print("IPK   : ");
        double ipk = Double.parseDouble(sc.nextLine());

        Mahasiswa08 m = new Mahasiswa08(nim, nama, kelas, ipk);
        sll.addLast(m);

        System.out.println("\nIsi Linked List:");
        sll.print();

        System.out.print("\nTambah data lagi? (y/n): ");
        lanjut = sc.nextLine();
    }

    System.out.println("Program selesai.");
    sc.close();
```

```
    }
  }
```

PERCOBAAN 2

```
n' 'PERTEMUAN11.SLLMain08'

Masukkan data mahasiswa
NIM   : 21212203
Nama  : Dirga
Kelas : 4D
IPK   : 3.6

Tambah data lagi? (y/n): y

Masukkan data mahasiswa
NIM   : 22212202
Nama  : Cintia
Kelas : 3D
IPK   : 3.5

Tambah data lagi? (y/n): y

Masukkan data mahasiswa
NIM   : 23212201
Nama  : Bimon
Kelas : 2B
IPK   : 3.8

Tambah data lagi? (y/n): y

Masukkan data mahasiswa
NIM   : 24212200
Nama  : Alvaro
Kelas : 1A
IPK   : 4.0

Tambah data lagi? (y/n): n
data index 1 :
Cintia    22212202       3D      3.5
data mahasiswa an Bimon berada pada index : 2

Isi Linked List:
Cintia    22212202       3D      3.5
Bimon     23212201       2B      3.8

Isi Linked List:
Bimon     23212201       2B      3.8

Program selesai.
PS D:\Algoritma-dan-Struktur-Data>
```

Kode Program Mahasiswa08 :

```
package PERTEMUAN11;


public class Mahasiswa08 {
    String nama;
    String nim;
    String kelas;
    double ipk;


    Mahasiswa08() {


    }


    Mahasiswa08(String nm, String name, String kls, double ip) {
        nama = name;
        nim = nm;
        kelas = kls;
        ipk = ip;
    }


    void tampilInformasi() {
        System.out.println(nama + "\t " + nim + "\t" + kelas + "\t" + ipk);
    }
}
```

Kode Porgram NodeMahasiswa08 :

```
package PERTEMUAN11;

public class NodeMahasiswa08 {
    Mahasiswa08 data;
    NodeMahasiswa08 next;

    public NodeMahasiswa08(Mahasiswa08 data, NodeMahasiswa08 next) {
        this.data = data;
        this.next = next;
    }
}
```

Kode Program singleLinkedList08 :

```
package PERTEMUAN11;

public class SingleLinkedList08 {
    NodeMahasiswa08 head;
    NodeMahasiswa08 tail;

    boolean IsEmpty() {
        return (head == null);
    }
```

```
public void print() {
    if (!IsEmpty()) {
        NodeMahasiswa08 tmp = head;
        System.out.println("Isi Linked List:\t");
        while (tmp != null) {
            tmp.data.tampilInformasi();
            tmp = tmp.next;
        }
        System.out.println("");
    } else {
        System.out.println("Linked List Kosong");
    }
}

public void addFirst(Mahasiswa08 input) {
    NodeMahasiswa08 ndInput = new NodeMahasiswa08(input, null);
    if (IsEmpty()) {
        head = ndInput;
        tail = ndInput;
    } else {
        ndInput.next = head;
        head = ndInput;
    }
```

```
}

public void addLast(Mahasiswa08 input) {
    NodeMahasiswa08 ndInput = new NodeMahasiswa08(input, null);
    if (IsEmpty()) {
        head = ndInput;
        tail = ndInput;
    } else {
        tail.next = ndInput;
        tail = ndInput;
    }
}

public void insertAfter(String key, Mahasiswa08 input) {
    NodeMahasiswa08 ndInput = new NodeMahasiswa08(input, null);
    NodeMahasiswa08 tmp = head;
    do {
        if (tmp.data.nama.equalsIgnoreCase(key)) {
            ndInput.next = tmp.next;
            tmp.next = ndInput;
            if (ndInput.next == null) {
                tail = ndInput;
            }
            break;
```

```
        }
        tmp = tmp.next;
    } while (tmp != null);
}

public void insertAt(int index, Mahasiswa08 input) {
    if (index < 0) {
        System.out.println("indeks salah");
    } else if (index == 0) {
        addFirst(input);
    } else {
        NodeMahasiswa08 temp = head;
        for (int i = 0; i < index - 1; i++) {
            temp = temp.next;
        }
        temp.next = new NodeMahasiswa08(input, temp.next);
        if (temp.next.next == null) {
            tail = temp.next;
        }
    }
}

public void getData(int index) {
    NodeMahasiswa08 tmp = head;
```

```
        for (int i = 0; i < index; i++) {
            tmp = tmp.next;
        }
        tmp.data.tampilInformasi();
    }

    public int indexOf(String key) {
        NodeMahasiswa08 tmp = head;
        int index = 0;
        while (tmp != null && !tmp.data.nama.equalsIgnoreCase(key)) {
            tmp = tmp.next;
            index++;
        }

        if (tmp == null) {
            return -1;
        } else {
            return index;
        }
    }

    public void removeFirst() {
        if (IsEmpty()) {
            System.out.println("Linked List masih kosong, tidak dapat dihapus!");
```

```
    } else if (head == tail) {
        head = tail = null;
    } else {
        head = head.next;
    }
}

public void removeLast() {
    if (IsEmpty()) {
        System.out.println("Linked List masih kosong, tidak dapat dihapus!");
    } else if (head == tail) {
        head = tail = null;
    } else {
        NodeMahasiswa08 temp = head;
        while (temp.next != tail) {
            temp = temp.next;
        }
        temp.next = null;
        tail = temp;
    }
}

public void remove(String key) {
    if (IsEmpty()) {
```

```
            System.out.println("Linked List masih kosong, tidak dapat dihapus!");
        } else {
            NodeMahasiswa08 temp = head;
            while (temp != null) {
                if ((temp.data.nama.equalsIgnoreCase(key)) && (temp == head)) {
                    this.removeFirst();
                    break;
                } else if (temp.data.nama.equalsIgnoreCase(key)) {
                    temp.next = temp.next.next;
                    if (temp.next == null) {
                        tail = temp;
                    }
                    break;
                }
                temp = temp.next;
            }
        }
    }

    public void removeAt(int index) {
        if (index == 0) {
            removeFirst();
        } else {
            NodeMahasiswa08 temp = head;
```

```
            for (int i = 0; i < index - 1; i++) {
                temp = temp.next;
            }
            temp.next = temp.next.next;
            if (temp.next == null) {
                tail = temp;
            }
        }
    }
}
```

Kode Program SLLMain08 :

```
package PERTEMUAN11;

import java.util.Scanner;

public class SLLMain08 {
    public static void main(String[] args) {
        SingleLinkedList08 sll = new SingleLinkedList08();
        Scanner sc = new Scanner(System.in);

        String lanjut = "y";

        while (lanjut.equalsIgnoreCase("y")) {
```

```
    System.out.println("\nMasukkan data mahasiswa");

    System.out.print("NIM   : ");
    String nim = sc.nextLine();

    System.out.print("Nama  : ");
    String nama = sc.nextLine();

    System.out.print("Kelas : ");
    String kelas = sc.nextLine();

    System.out.print("IPK   : ");
    double ipk = Double.parseDouble(sc.nextLine());

    Mahasiswa08 m = new Mahasiswa08(nim, nama, kelas, ipk);
    sll.addLast(m);

    System.out.print("\nTambah data lagi? (y/n): ");
    lanjut = sc.nextLine();
}

System.out.println("data index 1 : ");
    sll.getData(1);
```

```java
            System.out.println("data mahasiswa an Bimon berada pada index : "+sll.indexOf("bimon"));
        System.out.println();


        sll.removeFirst();
        sll.removeLast();
        sll.print();
        sll.removeAt(0);
        sll.print();


        System.out.println("Program selesai.");
        sc.close();
    }
}
```

Pertanyaan

1. Mengapa digunakan keyword break pada fungsi remove? Jelaskan!

   Jawab : Keyword break digunakan untuk menghentikan perulangan (loop) ketika data yang ingin dihapus sudah ditemukan dan berhasil diproses.

   Pada method remove, program melakukan pencarian node berdasarkan nama. Saat node yang sesuai sudah ditemukan dan dihapus, maka tidak perlu lagi melanjutkan pencarian ke node berikutnya. Oleh karena itu, break digunakan agar loop langsung berhenti.

   Jika break tidak digunakan, maka program akan tetap melanjutkan perulangan hingga akhir linked list, yang bisa menyebabkan:

   - proses menjadi tidak efisien (membuang waktu),
   - bahkan berpotensi error jika manipulasi data terjadi lebih dari sekali.

   Jadi, fungsi utama break di sini adalah menghentikan proses setelah data berhasil dihapus agar lebih efisien dan aman.

2. Jelaskan kegunaan kode dibawah pada method remove

```
temp.next = temp.next.next;
            if (temp.next == null) {
                tail = temp;
            }
```

Jawab : Kode tersebut berfungsi untuk:

1. Menghapus node dari linked list dengan cara melewati node tersebut
2. Menjaga agar pointer tail tetap benar jika node terakhir dihapus

TUGAS

Kode Program Mhs08 :

```
package PERTEMUAN11;


public class Mhs08 {

    String nim;

    String nama;

    String kelas;


    public Mhs08(String nim, String nama, String kelas) {

        this.nim = nim;

        this.nama = nama;

        this.kelas = kelas;

    }


    public void tampil() {

        System.out.println(nim + " - " + nama + " - " + kelas);
```

```
    }
}
```

Kode Program LikedList08 :

```
package PERTEMUAN11;

public class LinkedList08 {
    Mhs08 data;
    LinkedList08 next;

    public LinkedList08 (Mhs08 data) {
        this.data = data;
        this.next = null;
    }
}
```

Kode Program QueueLinkedList08 :

```
package PERTEMUAN11;

public class QueueLinkedList08 {
    LinkedList08 front;
    LinkedList08 rear;
    int size;
```

```
public QueueLinkedList08() {
    front = rear = null;
    size = 0;
}

public boolean isEmpty() {
    return front == null;
}

public boolean isFull() {
    return false;
}

public void enqueue(Mhs08 mhs) {
    LinkedList08 newNode = new LinkedList08(mhs);

    if (isEmpty()) {
        front = rear = newNode;
    } else {
        rear.next = newNode;
        rear = newNode;
    }

    size++;
```

```
    System.out.println(mhs.nama + " masuk antrian");
}

public void dequeue() {
    if (isEmpty()) {
        System.out.println("Antrian kosong");
    } else {
        System.out.print("Memanggil: ");
        front.data.tampil();

        front = front.next;
        size--;

        if (front == null) {
            rear = null;
        }
    }
}

public void peekFront() {
    if (isEmpty()) {
        System.out.println("Antrian kosong");
    } else {
        System.out.print("Terdepan: ");
```

```
        front.data.tampil();
    }
}

public void peekRear() {
    if (isEmpty()) {
        System.out.println("Antrian kosong");
    } else {
        System.out.print("Terakhir: ");
        rear.data.tampil();
    }
}

public void print() {
    if (isEmpty()) {
        System.out.println("Antrian kosong");
    } else {
        LinkedList08 temp = front;
        System.out.println("Daftar antrian:");
        while (temp != null) {
            temp.data.tampil();
            temp = temp.next;
        }
    }
```

```
    }

    public int jumlah() {
        return size;
    }

    public void clear() {
        front = rear = null;
        size = 0;
        System.out.println("Antrian dikosongkan");
    }
}
```

Kode Program MainMenu08 :

```
package PERTEMUAN11;

import java.util.Scanner;

public class MainMenu08 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        QueueLinkedList08 antrian = new QueueLinkedList08();

        int pilih;
```

```
do {
    System.out.println("\n=== MENU ANTRIAN ===");
    System.out.println("1. Tambah Antrian");
    System.out.println("2. Panggil Antrian");
    System.out.println("3. Lihat Terdepan");
    System.out.println("4. Lihat Terakhir");
    System.out.println("5. Tampilkan Semua");
    System.out.println("6. Jumlah Antrian");
    System.out.println("7. Kosongkan Antrian");
    System.out.println("0. Keluar");
    System.out.print("Pilih: ");
    pilih = sc.nextInt();
    sc.nextLine();

    switch (pilih) {
        case 1:
            System.out.print("NIM   : ");
            String nim = sc.nextLine();
            System.out.print("Nama  : ");
            String nama = sc.nextLine();
            System.out.print("Kelas : ");
            String kelas = sc.nextLine();
```

```
    Mhs08 m = new Mhs08(nim, nama, kelas);

    antrian.enqueue(m);

    break;


case 2:

    antrian.dequeue();

    break;


case 3:

    antrian.peekFront();

    break;


case 4:

    antrian.peekRear();

    break;


case 5:

    antrian.print();

    break;


case 6:

    System.out.println("Jumlah antrian: " + antrian.jumlah());

    break;
```

```java
            case 7:
                antrian.clear();
                break;

            case 0:
                System.out.println("Terima kasih");
                break;

            default:
                System.out.println("Pilihan tidak valid");
          }

        } while (pilih != 0);

        sc.close();
    }
}
```